UNIVERSIDAD PERUANA UNIÓN

Facultad de Ingeniería y Arquitectura · EP Ingeniería de Sistemas

Asignatura: Desarrollo de Aplicaciones Móviles (DAM) · Ciclo VI / Semestre 2026-2

ACTIVIDAD AUTÓNOMA N.º 11

PRUEBAS UNITARIAS EN COMMONTEST Y EVIDENCIA DE ESCENARIOS OFFLINE-FIRST EN PHARMAMOBIL

Estudiante	Julio Fabián Rodríguez Bazán
Sesión	Sesión 11 — Persistencia local con SQLDelight y offline-first
Repositorio GitHub	https://github.com/fabianrodriguez33/Pharma-Mobil
Rama entregada	https://github.com/fabianrodriguez33/Pharma-Mobil/tree/feature/offline-first-rodriguez
Fecha de entrega	26/10/2026

1. Resumen ejecutivo

Esta actividad cierra la Unidad 2: valida con pruebas automatizadas y con escenarios reales en emulador que PharmaMobil es offline-first, agrega un indicador de la última sincronización y prepara la sustentación del Producto U2. Todo se probó contra PharmaSoft real (Spring Boot + Oracle) y un emulador Pixel 8 (Android API 37.1).

Producto	Entregable	Estado
1 · Pruebas unitarias (7 pts)	6 pruebas del enunciado + 2 de apoyo en ProductoRepositorioOfflineTest; 80 pruebas en el proyecto, 0 fallos	Completo
2 · Bitácora offline (5 pts)	5 escenarios ejecutados en el emulador con capturas, pasos y resultado	Completo
3 · Indicador de sincronización (4 pts)	Marca de tiempo persistida + texto en el encabezado + botón «Sincronizar»	Completo
4 · Guion de sustentación (4 pts)	Guion de 5 criterios, 8 minutos en total	Completo

Hallazgos técnicos durante la ejecución (corregidos antes de la entrega):

- Crash al abrir el catálogo: el estado del ViewModel se declaraba después del bloque init (NullPointerException).
- El backend limita la página a 100 productos; la descarga ahora recorre todas las páginas.
- La marca de sincronización exigió una migración SQLDelight (1.sqm, esquema v2); se verificó en el emulador que una base v1 ya instalada migra sin perder datos.

2. Producto 1 — Suite de pruebas unitarias en commonTest

Archivo: shared/src/commonTest/kotlin/pe/edu/upeu/pharmamobil/ProductoRepositorioOfflineTest.kt. Cada prueba crea su propia base con JdbcSqliteDriver(IN_MEMORY) y simula el servidor con MockEngine de Ktor, por lo que no comparten estado.

N.º	Prueba	Qué verifica	Resultado
1	prueba1_sincronizarConRedGuardaLosRemotos…	Con 200 OK la base guarda los productos remotos y observar() emite la lista completa	Pasa
2	prueba2_registrarSinRedGuardaIdNegativo…	Con 503 el producto se guarda con id < 0 y pendiente = 1; el Flow lo emite	Pasa
3	prueba3_alReconectarSubeElPendiente…	POST antes del GET; la fila queda con pendiente = 0 y el id definitivo del servidor	Pasa
4	prueba4_laBajadaNoBorraLoPendiente	reemplazarSincronizados() borra solo pendiente = 0 y conserva lo creado sin red	Pasa
5	prueba5_idsNegativosYPositivosConviven	Ids -1, -2 conviven con 1, 2 del servidor sin sobrescribirse	Pasa

6	prueba6_sincronizarDosVecesNoDuplica…	Dos sincronizaciones seguidas no duplican filas	Pasa
+	conElServidorFueraDeLinea…	Con el servidor caído, observar() emite lo guardado sin crash	Pasa
+	laSincronizacionExitosaGuardaSuMarca…	La marca de tiempo solo se guarda si la sincronización tuvo éxito	Pasa

Pruebas complementarias: TextoSincronizacionTest (5 pruebas del texto del indicador) y ProductoViewModelOfflineTest (el ViewModel refleja un alta sin recargar a mano).

Resultado por clase (extraído de los reportes JUnit)

```
pe.edu.upeu.pharmamobil.ProductoRepositorioOfflineTest  tests=8 fallos=0
    PASSED  laSincronizacionExitosaGuardaSuMarcaDeTiempoYLaFallidaNo  (3.083s)
    PASSED  prueba5_idsNegativosYPositivosConviven  (0.022s)
    PASSED  conElServidorFueraDeLineaObservarEmiteLosDatosGuardados  (0.037s)
    PASSED  prueba2_registrarSinRedGuardaIdNegativoPendienteYElFlowLoEmite  (0.068s)
    PASSED  prueba4_laBajadaNoBorraLoPendiente  (0.01s)
    PASSED  prueba3_alReconectarSubeElPendienteYQuedaConElIdDelServidor  (0.063s)
    PASSED  prueba1_sincronizarConRedGuardaLosRemotosYObservarLosEmite  (0.022s)
    PASSED  prueba6_sincronizarDosVecesNoDuplicaRegistros  (0.043s)
pe.edu.upeu.pharmamobil.SharedCommonTest  tests=1 fallos=0
pe.edu.upeu.pharmamobil.SharedLogicAndroidHostTest  tests=1 fallos=0
pe.edu.upeu.pharmamobil.data.mapper.ProductoMapperTest  tests=2 fallos=0
pe.edu.upeu.pharmamobil.data.repository.CerrarSesionTest  tests=1 fallos=0
pe.edu.upeu.pharmamobil.data.repository.ClienteRepositorioEnMemoriaTest  tests=2 fallos=0
pe.edu.upeu.pharmamobil.data.repository.ProductoRepositorioEnMemoriaTest  tests=2 fallos=0
pe.edu.upeu.pharmamobil.di.AppModuleTest  tests=4 fallos=0
pe.edu.upeu.pharmamobil.domain.error.ErrorApiTest  tests=4 fallos=0
pe.edu.upeu.pharmamobil.domain.model.ClienteTest  tests=4 fallos=0
pe.edu.upeu.pharmamobil.domain.model.ProductoTest  tests=5 fallos=0
pe.edu.upeu.pharmamobil.domain.usecase.ListarProductosUseCaseTest  tests=2 fallos=0
pe.edu.upeu.pharmamobil.domain.usecase.RegistrarClienteUseCaseTest  tests=8 fallos=0
pe.edu.upeu.pharmamobil.domain.usecase.RegistrarProductoUseCaseTest  tests=11 fallos=0
pe.edu.upeu.pharmamobil.presentation.cliente.ClienteViewModelTest  tests=4 fallos=0
pe.edu.upeu.pharmamobil.presentation.login.LoginViewModelTest  tests=4 fallos=0
pe.edu.upeu.pharmamobil.presentation.producto.ProductoViewModelOfflineTest  tests=1 fallos=0
    PASSED  elViewModelMuestraLoRegistradoSinRecargarAMano  (0.062s)
pe.edu.upeu.pharmamobil.presentation.producto.ProductoViewModelTest  tests=11 fallos=0
pe.edu.upeu.pharmamobil.presentation.productos.TextoSincronizacionTest  tests=5 fallos=0
    PASSED  sinSincronizacionPreviaDiceNuncaSincronizado  (0.004s)
    PASSED  enMinutosDiceHaceXMinutos  (0.001s)
    PASSED  deOtroDiaMuestraLaFecha  (0.001s)
    PASSED  elMismoDiaDiceHoyALasHoraMinuto  (0.001s)
    PASSED  menosDeUnMinutoDiceHaceUnMomento  (0.001s)
TOTAL pruebas: 80
```

Salida completa de ./gradlew testAndroidHostTest

Comando ejecutado en D:\DAM\pharmaMobil-master con --rerun-tasks (BUILD SUCCESSFUL). El registro HTTP que se ve corresponde a las peticiones simuladas por MockEngine.

```
Calculating task graph as no cached configuration is available for tasks: testAndroidHostTest


> Configure project :shared
w: Native task 'iosSimulatorArm64Test' is disabled
Task 'iosSimulatorArm64Test' for target 'ios_simulator_arm64' cannot run on the current host (windows-x86_64).
Reason: simulator tests require macOS
Solution: To suppress this warning, add 'kotlin.native.ignoreDisabledTargets=true' to gradle.properties.



> Task :shared:generateAndroidMainAssets UP-TO-DATE
> Task :shared:androidPreBuild UP-TO-DATE
> Task :shared:generateAndroidHostTestAssets UP-TO-DATE
```

```
> Task :shared:preAndroidHostTestBuild UP-TO-DATE
> Task :shared:preAndroidMainBuild UP-TO-DATE
> Task :shared:copyNonXmlValueResourcesForAndroidMain NO-SOURCE
> Task :shared:copyNonXmlValueResourcesForAndroidHostTest NO-SOURCE
> Task :shared:convertXmlValueResourcesForAndroidMain NO-SOURCE
> Task :shared:convertXmlValueResourcesForAndroidHostTest NO-SOURCE
> Task :shared:kmpPartiallyResolvedDependenciesChecker
> Task :shared:checkKotlinGradlePluginConfigurationErrors SKIPPED
> Task :shared:convertXmlValueResourcesForCommonTest NO-SOURCE
> Task :shared:copyNonXmlValueResourcesForCommonTest NO-SOURCE
> Task :shared:prepareComposeResourcesTaskForAndroidMain NO-SOURCE
> Task :shared:prepareComposeResourcesTaskForAndroidHostTest NO-SOURCE
> Task :shared:generateResourceAccessorsForAndroidHostTest NO-SOURCE
> Task :shared:generateResourceAccessorsForAndroidMain NO-SOURCE
> Task :shared:prepareComposeResourcesTaskForCommonTest NO-SOURCE
> Task :shared:generateResourceAccessorsForCommonTest NO-SOURCE
> Task :shared:processAndroidHostTestJavaRes NO-SOURCE
> Task :shared:generateAndroidMainEmptyResourceFiles
> Task :shared:copyNonXmlValueResourcesForCommonMain
> Task :shared:writeAndroidMainAarMetadata
> Task :shared:convertXmlValueResourcesForCommonMain NO-SOURCE
> Task :shared:androidJar
> Task :shared:prepareComposeResourcesTaskForCommonMain
> Task :shared:generateExpectResourceCollectorsForCommonMain
> Task :shared:generateComposeResClass
> Task :shared:generateAndroidHostTestResources
> Task :shared:copyAndroidMainComposeResourcesToAndroidAssets
> Task :shared:generateResourceAccessorsForCommonMain
> Task :shared:generateActualResourceCollectorsForAndroidMain
> Task :shared:mergeAndroidMainAssets
> Task :shared:processAndroidMainManifest
> Task :shared:packageAndroidMainResources
> Task :shared:compileAndroidMainLibraryResources
> Task :shared:mapAndroidHostTestSourceSetPaths
> Task :shared:mergeAndroidHostTestAssets
> Task :shared:parseAndroidMainLocalResources
> Task :shared:checkAndroidHostTestAarMetadata
> Task :shared:generateAndroidMainRFile
> Task :shared:processAndroidHostTestManifest


> Task :shared:generateCommonMainPharmaDbInterface
WARN: Attempt to load key 'ide.hide.excluded.files' for not yet loaded registry
WARN: Attempt to load key 'psi.sleep.in.validity.check' for not yet loaded registry
WARN: Attempt to load key 'psi.incremental.reparse.depth.limit' for not yet loaded registry


> Task :shared:mergeAndroidHostTestResources
> Task :shared:processAndroidHostTestResources
> Task :shared:packageAndroidHostTestForUnitTest
> Task :shared:generateAndroidHostTestConfig
> Task :shared:compileAndroidMain
w: file:///D:/DAM/pharmaMobil-master/shared/src/androidMain/kotlin/pe/edu/upeu/pharmamobil/platform/Formato.android.kt:8:56
'constructor(p0: String!,
w: file:///D:/DAM/pharmaMobil-master/shared/src/androidMain/kotlin/pe/edu/upeu/pharmamobil/platform/Formato.android.kt:16:40
'constructor(p0: String!,
w: file:///D:/DAM/pharmaMobil-master/shared/src/androidMain/kotlin/pe/edu/upeu/pharmamobil/platform/InfoDispositivo.android.kt:5:1
'expect'/'actual' c
w: file:///D:/DAM/pharmaMobil-master/shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobil/App.kt:103:13 'fun KoinContext(koin: Koin
= ..., content: Co
w: file:///D:/DAM/pharmaMobil-master/shared/src/commonMain/kotlin/pe/edu/upeu/pharmamobil/platform/InfoDispositivo.kt:7:1
'expect'/'actual' classes (i


> Task :shared:processAndroidMainJavaRes
> Task :shared:bundleAndroidMainClassesToCompileJar
> Task :shared:bundleAndroidMainClassesToRuntimeJar
> Task :shared:compileAndroidHostTest


> Task :shared:testAndroidHostTest


ProductoRepositorioOfflineTest > laSincronizacionExitosaGuardaSuMarcaDeTiempoYLaFallidaNo STANDARD_ERROR
    SLF4J(W): No SLF4J providers were found.
    SLF4J(W): Defaulting to no-operation (NOP) logger implementation
    SLF4J(W): See https://www.slf4j.org/codes.html#noProviders for further details.


ProductoRepositorioOfflineTest > laSincronizacionExitosaGuardaSuMarcaDeTiempoYLaFallidaNo STANDARD_OUT
    HttpClient: REQUEST: http://localhost:8080/api/v1/productos?pagina=0&tamanio=100
    METHOD: GET
    COMMON HEADERS
    -> Accept: application/json
```

```
    -> Content-Type: application/json
    CONTENT HEADERS
    -> Content-Length: 0
    HttpClient: RESPONSE: 503 Service Unavailable
    METHOD: GET
    FROM: http://localhost:8080/api/v1/productos?pagina=0&tamanio=100
    COMMON HEADERS
    HttpClient: REQUEST: http://localhost:8080/api/v1/productos?pagina=0&tamanio=100
    METHOD: GET
    COMMON HEADERS
    -> Accept: application/json
    -> Content-Type: application/json
    CONTENT HEADERS
    -> Content-Length: 0
    HttpClient: RESPONSE: 200 OK
    METHOD: GET
    FROM: http://localhost:8080/api/v1/productos?pagina=0&tamanio=100
    COMMON HEADERS
    -> Content-Type: application/json


ProductoRepositorioOfflineTest > conElServidorFueraDeLineaObservarEmiteLosDatosGuardados STANDARD_OUT
    HttpClient: REQUEST: http://localhost:8080/api/v1/productos?pagina=0&tamanio=100
    METHOD: GET
    COMMON HEADERS
    -> Accept: application/json
    -> Content-Type: application/json
    CONTENT HEADERS
    -> Content-Length: 0
    HttpClient: RESPONSE: 503 Service Unavailable
    METHOD: GET
    FROM: http://localhost:8080/api/v1/productos?pagina=0&tamanio=100
    COMMON HEADERS


ProductoRepositorioOfflineTest > prueba2_registrarSinRedGuardaIdNegativoPendienteYElFlowLoEmite STANDARD_OUT
    HttpClient: REQUEST: http://localhost:8080/api/v1/productos
    METHOD: POST
    COMMON HEADERS
    -> Accept: application/json
    CONTENT HEADERS
    -> Content-Length: 109
    -> Content-Type: application/json
    HttpClient: RESPONSE: 503 Service Unavailable
    METHOD: POST
    FROM: http://localhost:8080/api/v1/productos
    COMMON HEADERS


ProductoRepositorioOfflineTest > prueba3_alReconectarSubeElPendienteYQuedaConElIdDelServidor STANDARD_OUT
    HttpClient: REQUEST: http://localhost:8080/api/v1/productos
    METHOD: POST
    COMMON HEADERS
    -> Accept: application/json
    CONTENT HEADERS
    -> Content-Length: 109
    -> Content-Type: application/json
    HttpClient: RESPONSE: 503 Service Unavailable
    METHOD: POST
    FROM: http://localhost:8080/api/v1/productos
    COMMON HEADERS
    HttpClient: REQUEST: http://localhost:8080/api/v1/productos
    METHOD: POST
    COMMON HEADERS
    -> Accept: application/json
    CONTENT HEADERS
    -> Content-Length: 109
    -> Content-Type: application/json
    HttpClient: RESPONSE: 201 Created
    METHOD: POST
    FROM: http://localhost:8080/api/v1/productos
    COMMON HEADERS
    -> Content-Type: application/json
    HttpClient: REQUEST: http://localhost:8080/api/v1/productos?pagina=0&tamanio=100
    METHOD: GET
    COMMON HEADERS
    -> Accept: application/json
    -> Content-Type: application/json
    CONTENT HEADERS
    -> Content-Length: 0
    HttpClient: RESPONSE: 200 OK
    METHOD: GET
    FROM: http://localhost:8080/api/v1/productos?pagina=0&tamanio=100
```

```
    COMMON HEADERS
    -> Content-Type: application/json


ProductoRepositorioOfflineTest > prueba1_sincronizarConRedGuardaLosRemotosYObservarLosEmite STANDARD_OUT
    HttpClient: REQUEST: http://localhost:8080/api/v1/productos?pagina=0&tamanio=100
    METHOD: GET
    COMMON HEADERS
    -> Accept: application/json
    -> Content-Type: application/json
    CONTENT HEADERS
    -> Content-Length: 0
    HttpClient: RESPONSE: 200 OK
    METHOD: GET
    FROM: http://localhost:8080/api/v1/productos?pagina=0&tamanio=100
    COMMON HEADERS
    -> Content-Type: application/json


ProductoRepositorioOfflineTest > prueba6_sincronizarDosVecesNoDuplicaRegistros STANDARD_OUT
    HttpClient: REQUEST: http://localhost:8080/api/v1/productos?pagina=0&tamanio=100
    METHOD: GET
    COMMON HEADERS
    -> Accept: application/json
    -> Content-Type: application/json
    CONTENT HEADERS
    -> Content-Length: 0
    HttpClient: RESPONSE: 200 OK
    METHOD: GET
    FROM: http://localhost:8080/api/v1/productos?pagina=0&tamanio=100
    COMMON HEADERS
    -> Content-Type: application/json
    HttpClient: REQUEST: http://localhost:8080/api/v1/productos?pagina=0&tamanio=100
    METHOD: GET
    COMMON HEADERS
    -> Accept: application/json
    -> Content-Type: application/json
    CONTENT HEADERS
    -> Content-Length: 0
    HttpClient: RESPONSE: 200 OK
    METHOD: GET
    FROM: http://localhost:8080/api/v1/productos?pagina=0&tamanio=100
    COMMON HEADERS
    -> Content-Type: application/json


CerrarSesionTest > cerrarSesionAplicaLosCuatroEfectos STANDARD_OUT
    HttpClient: REQUEST: http://localhost/api/v1/auth/login
    METHOD: POST
    COMMON HEADERS
    -> Accept: application/json
    CONTENT HEADERS
    -> Content-Length: 55
    -> Content-Type: application/json
    HttpClient: RESPONSE: 200 OK
    METHOD: POST
    FROM: http://localhost/api/v1/auth/login
    COMMON HEADERS
    -> Content-Type: application/json
    HttpClient: REQUEST: http://localhost/api/v1/productos
    METHOD: GET
    COMMON HEADERS
    -> Accept: application/json
    -> Authorization: ***
    -> Content-Type: application/json
    CONTENT HEADERS
    -> Content-Length: 0
    HttpClient: RESPONSE: 200 OK
    METHOD: GET
    FROM: http://localhost/api/v1/productos
    COMMON HEADERS
    -> Content-Type: application/json
    HttpClient: REQUEST: http://localhost/api/v1/productos
    METHOD: GET
    COMMON HEADERS
    -> Accept: application/json
    -> Content-Type: application/json
    CONTENT HEADERS
    -> Content-Length: 0
    HttpClient: RESPONSE: 200 OK
    METHOD: GET
    FROM: http://localhost/api/v1/productos
    COMMON HEADERS
```

```
    -> Content-Type: application/json


ProductoViewModelOfflineTest > elViewModelMuestraLoRegistradoSinRecargarAMano STANDARD_OUT
    HttpClient: REQUEST: http://localhost:8080/api/v1/productos
    METHOD: POST
    COMMON HEADERS
    -> Accept: application/json
    CONTENT HEADERS
    -> Content-Length: 109
    -> Content-Type: application/json
    HttpClient: RESPONSE: 503 Service Unavailable
    METHOD: POST
    FROM: http://localhost:8080/api/v1/productos
    COMMON HEADERS


[Incubating] Problems report is available at: file:///D:/DAM/pharmaMobil-master/build/reports/problems/problems-report.html


BUILD SUCCESSFUL in 1m 52s
33 actionable tasks: 33 executed
Configuration cache entry stored.
```

3. Producto 2 — Bitácora de 5 escenarios offline (emulador Pixel 8)

Entorno: emulador Pixel 8 (Android API 37.1), PharmaSoft en localhost:8080 (Oracle), usuario admin del seed del proyecto. Modo Avión activado con «adb shell cmd connectivity airplane-mode». La base local se inspeccionó extrayendo pharma.db con «adb exec-out run-as … cat databases/pharma.db».

Escenario 1 — Apertura sin red con datos guardados

Pasos:

- Con datos ya sincronizados, activar Modo Avión y cerrar la app por completo (force-stop).
- Abrir PharmaMobil y entrar a Catálogo.

Resultado observado: el catálogo se lista desde SQLite (3 productos) y aparece el aviso «Sin conexión: mostrando datos guardados» junto con la antigüedad de los datos. No hay pantalla en blanco ni error.

Estado de la base local: filas sincronizadas ids 1, 43 y 44, todas con pendiente = 0 (las mismas que muestra la extracción del Escenario 2, sin la fila -1), más la marca de la última sincronización.

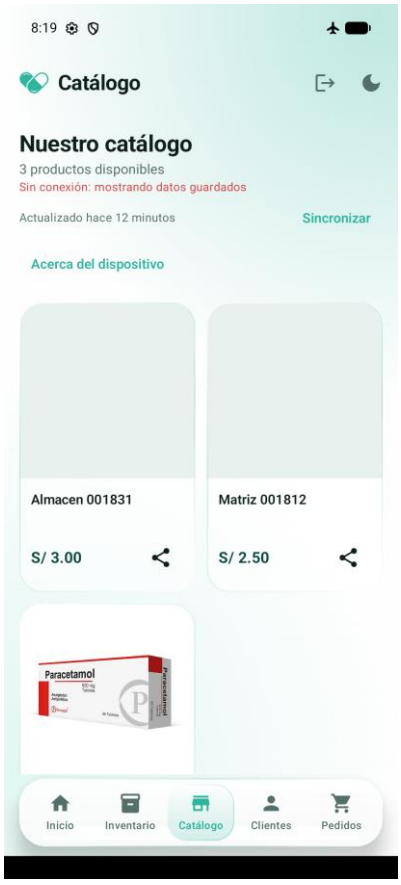


Figura 1. Catálogo en Modo Avión con aviso y «Actualizado hace 12 minutos».

Conclusión: La pantalla no depende de la red para existir: lee de pharma.db y el servidor solo la refresca. Lo importante es que el fallo de sincronización se comunica (aviso en rojo) sin vaciar el catálogo.

Escenario 2 — Registro en Modo Avión

Pasos:

- Con Modo Avión activo, ir a Inventario y registrar «Amoxicilina_Offline», S/ 6.80, 30 unidades.

Resultado observado: aparece de inmediato en la lista (4 productos) sin recargar. En SQLite quedó con id negativo y pendiente = 1:

```
sqlite> select id, nombre, precio, stock, pendiente from producto order by id;
-1 | Amoxicilina_Offline | 6.8 | 30 | 1
1 | Paracetamol 500mg | 12.5 | 40 | 0
43 | Matriz 001812 | 2.5 | 10 | 0
44 | Almacen 001831 | 3.0 | 5 | 0
```



Figura 2. Producto registrado sin red y visible al instante.

Conclusión: El alta se confirma contra la base local, no contra el servidor: el usuario no espera ni pierde su trabajo. El id negativo es lo que permite distinguir lo local de lo definitivo sin chocar con los ids de PharmaSoft.

Escenario 3 — Reconexión y sincronización exitosa

Pasos:

- Antes de reconectar se editó el pendiente a S/ 7.20 (ver Escenario 5).
- Desactivar Modo Avión y abrir Catálogo: la app sincroniza al entrar; se pulsó además «Sincronizar».
- Verificar en SQLite y en Swagger UI (GET /api/v1/productos).

Resultado observado: el POST se envió, la fila -1 pasó a id 48 con pendiente = 0 y conservó el precio editado offline (7.20). Swagger UI muestra el producto con id 48 y estado true.

```
sqlite> select id, nombre, precio, stock, pendiente from producto order by id;
1 | Paracetamol 500mg | 12.5 | 40 | 0
43 | Matriz 001812 | 2.5 | 10 | 0
44 | Almacen 001831 | 3.0 | 5 | 0
48 | Amoxicilina_Offline | 7.2 | 30 | 0
```

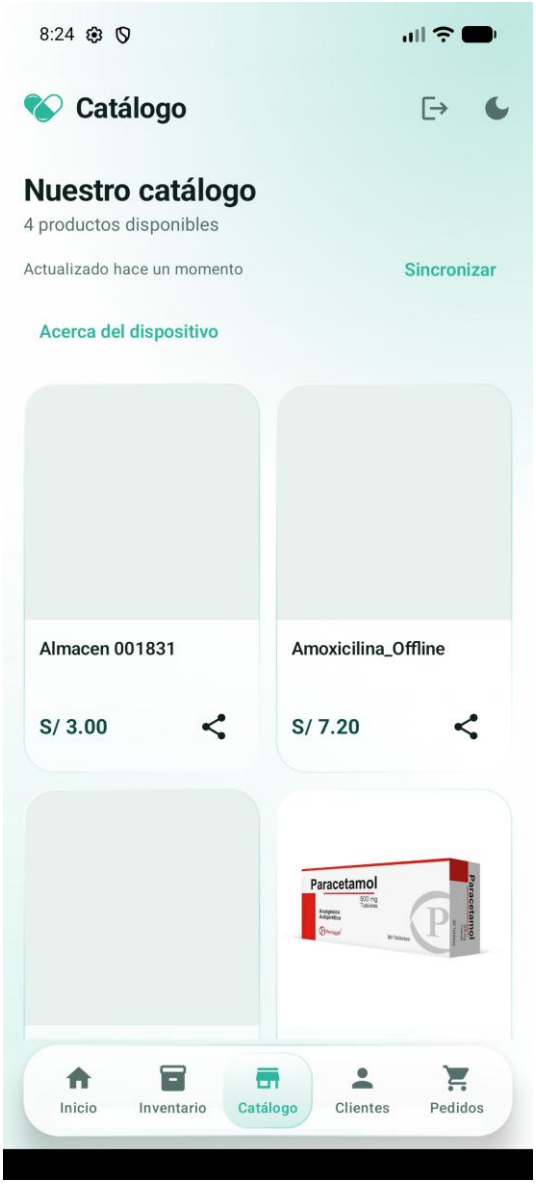


Figura 3. Tras reconectar: «Actualizado hace un momento».

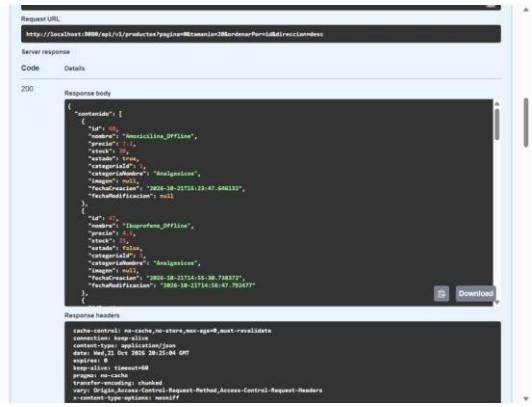


Figura 4. Swagger UI: GET /api/v1/productos (200) con id 48.

Conclusión: El orden «subir antes de bajar» se cumple en la práctica: el pendiente llegó al servidor con su edición incluida y la bajada posterior no lo borró. Swagger confirma que el dato existe del lado del servidor, no solo en el teléfono.

Escenario 4 — Sincronización con token vencido

Para provocar un vencimiento real se reinició PharmaSoft con app.jwt.expiracion-minutos = 1 (el login devolvió expiraEnSegundos = 60).

Pasos:

- Iniciar sesión (token de 60 s), activar Modo Avión y registrar «Token_Vencido» (queda pendiente).
- Esperar más de 60 s para que el JWT venza; desactivar Modo Avión y abrir Catálogo (sincroniza).
- Observar el 401, comprobar la base y volver a iniciar sesión.

Resultado observado: el servidor respondió 401 (registrado en Logcat), la app cerró la sesión y mostró «Tu sesión expiró. Inicia sesión nuevamente». El trabajo pendiente NO se perdió; tras reingresar se envió y la fila -1 pasó a id 49 con pendiente = 0.

Momento	Contenido de la tabla producto (id | nombre | pendiente)
Antes del 401	-1 | Token_Vencido | 1 (pendiente, creado en Modo Avión)
Tras el 401 (sesión cerrada)	-1 | Token_Vencido | 1 → intacto
Tras reingresar y sincronizar	49 | Token_Vencido | 0 → enviado



Figura 5. Aviso de sesión vencida tras el 401.

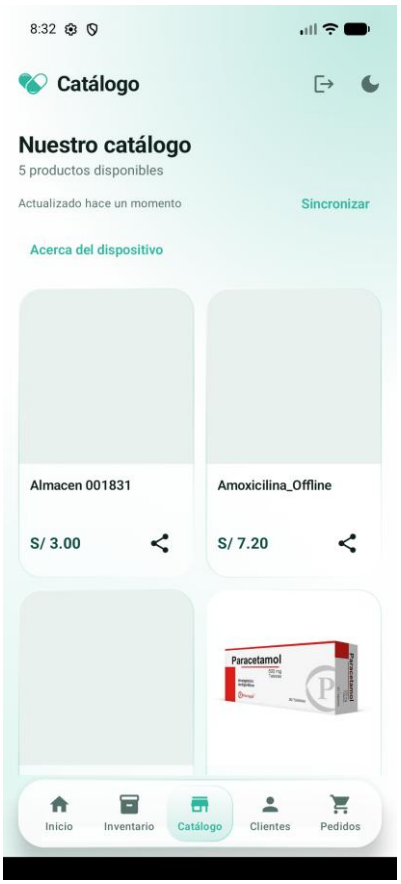


Figura 6. Tras reingresar: pendiente enviado y catálogo actualizado.

Estado de la base local: las extracciones de la tabla anterior (antes del 401, tras el 401 y tras reingresar) se tomaron con «adb run-as … cat databases/pharma.db».

Diseño: el cierre de sesión solo borra el token y el perfil (AlmacenSeguro); la base pharma.db no se toca, por eso lo pendiente sobrevive. Además, si el 401 llegara a la pantalla de catálogo el aviso diría «Sesión vencida… tus cambios pendientes siguen guardados».

Conclusión: Un 401 es un problema de identidad, no de datos: la app pide reautenticar pero jamás toca pharma.db. Separar la sesión (AlmacenSeguro) de los datos (SQLite) es lo que garantiza que el trabajo pendiente sobreviva a un token vencido.

Escenario 5 — Edición y eliminación sin red

Pasos y resultados (Modo Avión activo):

Acción	Resultado observado
Eliminar un producto ya sincronizado (Almacen 001831)	Mensaje «Sin conexión con el servidor»; el producto permanece; nada cambia
Editar un producto ya sincronizado (S/ 3.00 → 3.50)	Mismo mensaje; el precio sigue en S/ 3.00
Editar un producto pendiente (Amoxicilina_Offline 6.80 → 7.20)	Se aplica al instante en local; al reconectar viaja en el POST (el servidor guardó 7.20)
Eliminar un producto pendiente (Esc5_Pendiente)	«Producto eliminado correctamente»; la fila desaparece de SQLite y nunca llega al servidor

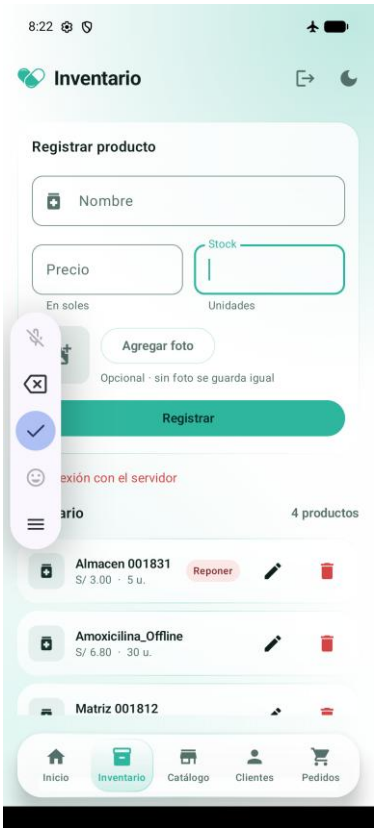


Figura 7. Eliminar un registro sincronizado sin red.



Figura 8. Editar un registro sincronizado sin red.



Figura 9. Editar un pendiente: se aplica en local (S/ 7.20).

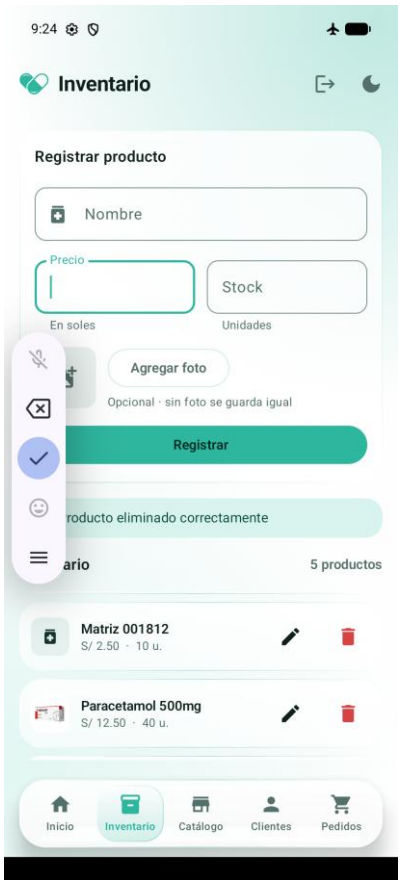


Figura 10. Eliminar un pendiente sin red.

Estado de la base local en cada paso (extraído de pharma.db):

a) Se crea el pendiente Esc5_Pendiente en Modo Avión

```
sqlite> select id, nombre, precio, stock, pendiente from producto order by id;
-1 | Esc5_Pendiente | 5.0 | 10 | 1
1 | Paracetamol 500mg | 12.5 | 40 | 0
43 | Matriz 001812 | 2.5 | 10 | 0
44 | Almacen 001831 | 3.0 | 5 | 0
48 | Amoxicilina_Offline | 7.2 | 30 | 0
49 | Token_Vencido | 9.9 | 15 | 0
```

b) Tras intentar eliminar el sincronizado «Almacen 001831»: nada cambia

```
sqlite> select id, nombre, precio, stock, pendiente from producto order by id;
-1 | Esc5_Pendiente | 5.0 | 10 | 1
1 | Paracetamol 500mg | 12.5 | 40 | 0
43 | Matriz 001812 | 2.5 | 10 | 0
44 | Almacen 001831 | 3.0 | 5 | 0
48 | Amoxicilina_Offline | 7.2 | 30 | 0
49 | Token_Vencido | 9.9 | 15 | 0
```

c) Tras editar el pendiente (5.0 → 5.5)

```
sqlite> select id, nombre, precio, stock, pendiente from producto order by id;
-1 | Esc5_Pendiente | 5.5 | 10 | 1
1 | Paracetamol 500mg | 12.5 | 40 | 0
43 | Matriz 001812 | 2.5 | 10 | 0
44 | Almacen 001831 | 3.0 | 5 | 0
```

```
48 | Amoxicilina_Offline | 7.2 | 30 | 0
49 | Token_Vencido | 9.9 | 15 | 0
```

d) Tras eliminar el pendiente: la fila -1 desaparece

```
sqlite> select id, nombre, precio, stock, pendiente from producto order by id;
1 | Paracetamol 500mg | 12.5 | 40 | 0
43 | Matriz 001812 | 2.5 | 10 | 0
44 | Almacen 001831 | 3.0 | 5 | 0
48 | Amoxicilina_Offline | 7.2 | 30 | 0
49 | Token_Vencido | 9.9 | 15 | 0
```

Argumentación técnica de la estrategia:

- Registros pendientes (id negativo): solo existen en el dispositivo, así que editarlos o eliminarlos es una operación local sobre una fila con pendiente = 1; el POST posterior ya lleva el estado final. No hay conflicto posible.
- Registros ya sincronizados: la app aplica write-through. La modificación o la baja se envían primero al servidor y solo si este confirma se refleja en SQLite. Sin red se rechaza con un mensaje claro y la base no cambia.
- ¿Es correcto? Es seguro y consistente: no hay datos perdidos, ni estados que diverjan de PharmaSoft, y el usuario recibe retroalimentación. Pero no es offline-first completo para edición/baja: el usuario no puede trabajar esas operaciones sin red. Es una limitación consciente, no un defecto oculto.
- Mejora recomendada (fuera del alcance): una tabla outbox de operaciones pendientes (CREATE/UPDATE/DELETE) con lápidas para las bajas y resolución de conflictos por fechaModificacion/versión, ejecutada con el mismo orden «subir antes de bajar».
- Verificado en el emulador: eliminar un pendiente borra la fila local (id < 0) sin llamar a la red; el estado de la base se muestra arriba en los pasos a–d.

Conclusión: El comportamiento es coherente pero asimétrico: lo que nació en el teléfono se gobierna en el teléfono, y lo que nació en el servidor solo cambia con permiso del servidor. Lo considero correcto como política de consistencia (nunca hay datos perdidos ni divergentes) e incompleto como experiencia offline: la siguiente mejora real sería la cola de operaciones.

4. Producto 3 — Indicador de última sincronización

Implementación

- Persistencia: tabla sincronizacion(clave, valor) en la misma base SQLite con la marca (epoch millis, Long). Se guarda únicamente al terminar con éxito subir pendientes y descargar el catálogo. Se eligió la base y no AlmacenSeguro porque queda junto a los datos que describe, es observable con Flow y no exige cambiar la interfaz de la plataforma.
- Migración: SQLDelight 1.sqm (esquema v1 → v2). Verificado en el emulador: una base v1 ya instalada migró sin perder los productos y mostró «Nunca sincronizado».
- UI: el encabezado de ProductosScreen muestra el texto del indicador y el botón «Sincronizar» (deshabilitado mientras sincroniza). El texto se refresca cada 30 s.
- Reglas del texto: sin marca → «Nunca sincronizado»; < 1 min → «Actualizado hace un momento»; < 60 min → «Actualizado hace X minutos»; mismo día → «Actualizado hoy a las HH:mm»; otro día → «Actualizado el dd/MM a las HH:mm».

Evidencia

Para los estados con antigüedad se modificó la marca en pharma.db con la app cerrada y en Modo Avión (para que la app no re-sincronizara al abrir).

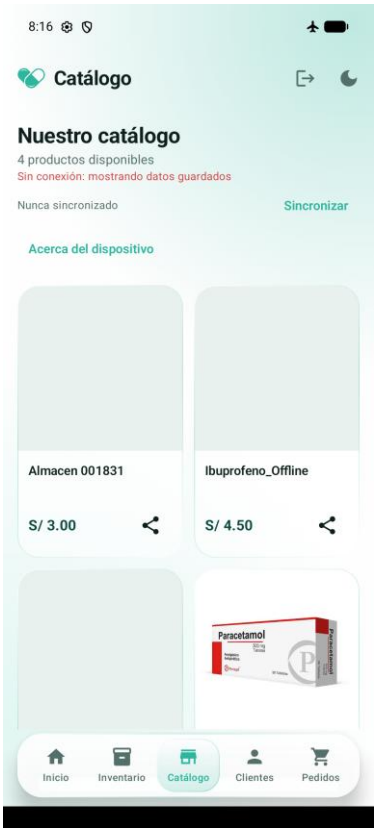


Figura 11. Estado inicial: «Nunca sincronizado».

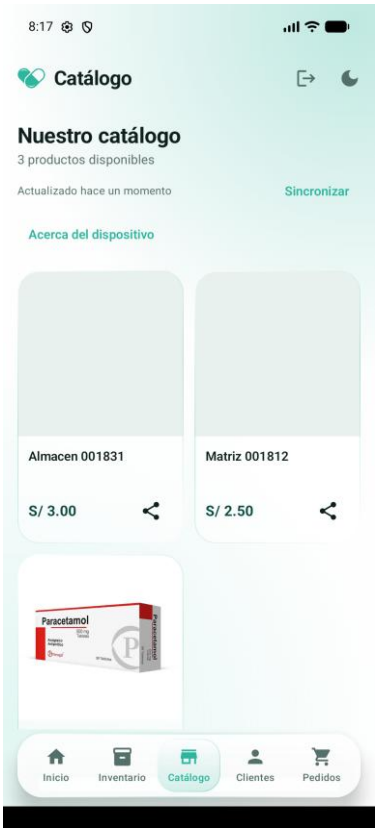


Figura 12. Datos recientes: «hace un momento».

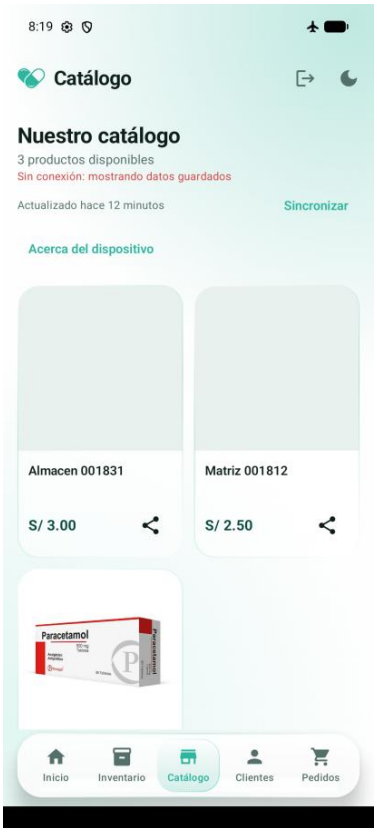


Figura 13. «Actualizado hace 12 minutos».

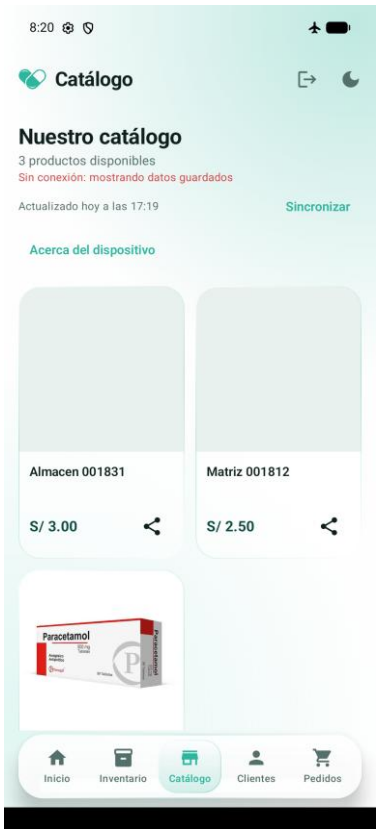


Figura 14. «Actualizado hoy a las 17:19» (3 h).

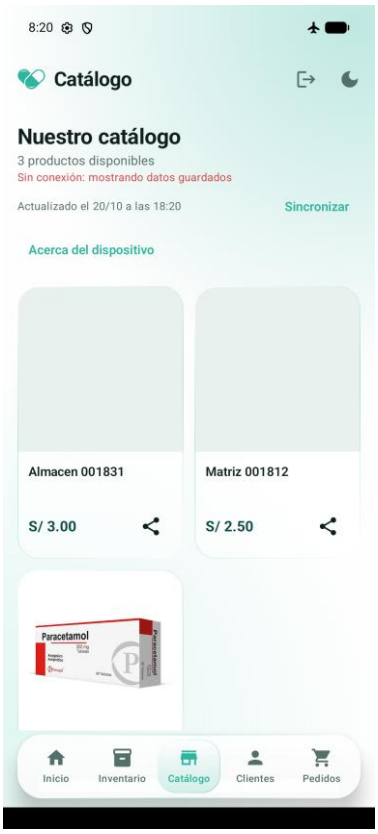


Figura 15. «Actualizado el 20/10 a las 18:20» (más de 1 día).

Conclusión: Guardar la marca junto a los datos hace que el indicador nunca mienta: se actualiza en la misma transacción lógica que la sincronización exitosa y un intento fallido no la mueve. El caso «Nunca sincronizado» evita mostrar una fecha vacía o inventada.

5. Producto 4 — Guion de sustentación del Producto U2 (27 de octubre)

Duración total: 8 minutos (1,5 + 1,5 + 1,5 + 1,5 + 2). Preparación previa: PharmaSoft y emulador encendidos, sesión cerrada, Swagger UI abierto, y datos sincronizados al menos una vez.

Criterio 1 — Consumo REST con Ktor (Sesión 7) (1,5 min)

Mostrar:

- HttpClientFactory.kt: un solo HttpClient con ContentNegotiation (JSON) y Logging con el token oculto.
- Motores por plataforma en PlatformModule: OkHttp (Android) y Darwin (iOS).
- ProductoDto/ProductoRequestDto y el log del GET /api/v1/productos con 200.

Decir:

- El motor se inyecta con Koin: la lógica compartida no sabe qué motor usa.
- Los DTO viven en data/remote y el dominio no los conoce (mapper).

Criterio 2 — CRUD completo (Sesión 8) (1,5 min)

Mostrar:

- Swagger UI o la app: GET lista, GET por id, POST, PUT y DELETE sobre /api/v1/productos.
- DELETE devuelve 204 No Content: ProductoApi.eliminar no intenta leer el cuerpo.
- Registrar, editar y eliminar un producto desde Inventario.

Decir:

- Los cinco verbos están implementados y probados.
- El backend hace baja lógica (estado = false); el mapper filtra los inactivos.

Criterio 3 — Estados y errores (Sesión 8) (1,5 min)

Mostrar:

- ProductoUiState: Fase (Cargando, SinProductos, ConProductos, Error) separada de Operacion (Inactiva, EnCurso, Fallida).
- Enviar el formulario vacío: el 400 se traduce a errores bajo cada campo (ErrorApi.Validacion).

Decir:

- Separar Fase de Operacion evita que una operación fallida borre la lista.
- ErrorApi es el único tipo de error que cruza a presentation; no expone Ktor.

Criterio 4 — Autenticación JWT (Sesión 10) (1,5 min)

Mostrar:

- Login con admin; token guardado cifrado en AlmacenSeguro (AES-GCM / Keychain).
- Plugin Auth de Ktor agrega la cabecera Bearer; el log muestra «Authorization: ***».
- Roles: admin ve todo; vendedor sin formulario de productos; almacen con catálogo escritura.
- Un 401 devuelve al login con aviso.

Decir:

- El token nunca se imprime ni se guarda en claro.
- Un 403 mantiene la sesión; un 401 la cierra.

Criterio 5 — Persistencia offline-first (Sesión 11) (2 min)

Mostrar:

- Modo Avión → abrir Catálogo: lista desde SQLite con «Sin conexión…» y «Actualizado hace X».
- Registrar un producto sin red: aparece al instante (id negativo, pendiente = 1).
- Reconectar y pulsar «Sincronizar»: llega a Swagger con id real.
- Código: ProductoRepositorioOffline.sincronizar() — subirPendientes() antes de descargar.

Decir:

- SQLite es la única fuente de verdad; la pantalla solo observa un Flow.
- El orden es sagrado: subir antes de bajar; si se invierte, la descarga borraría lo creado sin red.
- Los ids negativos evitan colisiones con los del servidor.
- Limitación conocida: editar/eliminar lo ya sincronizado requiere red (propuesta: outbox).

Plan de contingencia

- Si el backend no responde: usar la grabación/capturas del informe y mostrar las pruebas pasando (./gradlew testAndroidHostTest).
- Si el emulador va lento: ejecutar el ciclo con las pruebas 2, 3 y 4 de ProductoRepositorioOfflineTest.
- Pregunta probable: «¿Por qué no usar AlmacenSeguro para la fecha?» → la marca vive junto a los datos que describe.

6. Conclusiones y limitaciones

- PharmaMobil abre y lista el catálogo sin red, registra offline, y sincroniza en el orden correcto con id definitivo.
- Las 6 pruebas del enunciado y la suite completa (80) pasan; el build termina en BUILD SUCCESSFUL.
- Limitaciones: edición/baja de registros ya sincronizados sin red; iOS no se compiló ni ejecutó (entorno Windows); la foto de un producto creado sin red solo puede subirse tras sincronizar.

7. Lista de cotejo previa a la entrega

N.º	Criterio observable	Cumple
1	Documento en PDF nombrado S11_ActividadAutonoma_RodriguezBazan.pdf	Sí
2	Portada con nombre completo, sesión, fecha y enlace a la rama	Sí
3	Las seis pruebas están en commonTest y todas pasan	Sí
4	Se adjunta la salida completa de la ejecución de las pruebas	Sí
5	La bitácora contiene los cinco escenarios con su captura	Sí
6	El escenario 3 incluye la evidencia tomada desde Swagger	Sí (Figura 4)
7	El escenario 5 argumenta si el comportamiento es el correcto	Sí
8	La pantalla muestra cuándo fue la última sincronización correcta	Sí (Figuras 11–15)
9	Se resolvió el caso de una aplicación que nunca ha sincronizado	Sí (Figura 11)

10	El guion cubre los cinco criterios del Producto U2	Sí
11	Los commits están en mi rama y son accesibles desde el enlace entregado	Sí